HTML Notes

HyperText Markup Language

Student Study Notes | Syntax + Explanation + Examples

Contents

1. Introduction to HTML
2. Basics of HTML Elements
3. Semantic HTML
4. HTML Attributes
5. Headings
6. Paragraphs and Text Formatting
7. Tables
8. Dropdown (Select List)
9. Quotations
10. Lists
11. Block Elements: div and span
12. Layout
13. Responsive Web Design
14. iframes
15. The Head Element
16. Entities and URI / URL Encoding
17. Frames (Old HTML)
18. Charset (Character Encoding)
19. Forms
20. Form Elements
21. HTML Security

1. Introduction to HTML

HTML (HyperText Markup Language) is the standard language used to create web pages. **Hypertext** means text that contains links to other pages. **Markup** means we use **tags** to describe the structure of content (headings, paragraphs, images, links, etc.). HTML is not a programming language; it only describes structure.

- Created by **Tim Berners-Lee** in 1991. The latest standard is **HTML5**.
- Files are saved with the extension **.html** or **.htm** and opened in a web browser.
- Browsers read the tags and display the formatted page.
- You only need a text editor (Notepad, VS Code) and a browser to start.

Basic structure of an HTML page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first web page.</p>
</body>
</html>
```

Part	Meaning
<!DOCTYPE html>	Tells the browser the page uses HTML5.
<html>	Root element; wraps the whole page. lang sets the language.
<head>	Information about the page (title, meta, links). Not shown on the page.
<body>	All visible content: text, images, links, tables, forms.

2. Basics of HTML Elements

An **HTML element** consists of a **start tag**, the **content**, and an **end tag**.

```
<tagname>  Content goes here  </tagname>

<p>This is a paragraph.</p>
```

- **Empty (void) elements** have no content and no end tag:
, <hr>, <img>, <input>, <meta>, <link>.
- **Nesting**: elements can be placed inside other elements. Close the inner tag before the outer tag.
- Tag names are not case sensitive, but always write them in **lowercase**.
- **Comments** are not displayed: write them as <!-- comment text -->.

```
<body>
  <p>This is <b>bold</b> text.</p>   <!-- correct nesting -->
  <br>                               <!-- empty element -->
</body>
```

Block-level vs Inline elements

Type	Behaviour and examples
Block-level	Starts on a new line and takes the full width. Examples: div, p, h1-h6, ul, ol, table, form, section.
Inline	Stays in the same line and takes only needed width. Examples: span, a, b, i, img, strong, em.

3. Semantic HTML

Semantic elements clearly describe their meaning to both the browser and the developer. For example, <header> says "this is the page header", while says nothing about its content.

Tag	Purpose
<header>	Top part of a page or section: logo, title, intro.
<nav>	Group of navigation links (menu).
<main>	Main unique content of the page (use once).
<section>	A thematic group of content, usually with a heading.
<article>	Independent content: blog post, news story.
<aside>	Side content: sidebar, ads, related links.
<footer>	Bottom part: copyright, contact, links.
<figure>	Image/diagram with an optional <figcaption> caption.
<details>	Expandable hidden details; <summary> is its visible title.

```
<header><h1>My Blog</h1></header>
<nav><a href="index.html">Home</a> | <a href="about.html">About</a></nav>
<main>
  <article>
    <h2>First Post</h2>
    <p>Article text...</p>
  </article>
  <aside>Related links</aside>
</main>
<footer>&copy; 2026 My Blog</footer>
```

- **Benefits**: better SEO, better accessibility (screen readers), cleaner and readable code.
- Non-semantic elements: and .

4. HTML Attributes

Attributes give extra information about an element. They are written inside the **start tag** as **name="value"** pairs.

```
<tagname attribute="value">content</tagname>

<a href="https://example.com" target="_blank" title="Visit site">Click</a>
<img src="photo.jpg" alt="A photo" width="300" height="200">
```

Attribute	Use
id	Unique identifier of one element (used by CSS/JS).
class	Group name; many elements can share one class.
style	Inline CSS styling.
title	Tooltip text shown on mouse hover.
href	Link address for <a>.
src	Source file for img, script, iframe, audio, video.
alt	Alternate text for an image (accessibility + SEO).
width / height	Size of images and other elements.
lang	Language of the content, e.g. lang="en".
target	_blank opens link in a new tab; _self (default) same tab.
disabled / readonly	Boolean attributes for form fields.

> **Note:** Always use quotes around values and lowercase attribute names. Attributes like id, class, style, title, lang are **global attributes** and work on every element.

5. Headings

HTML has **six levels of headings**: <h1> is the most important (largest) and <h6> is the least important (smallest).

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

- Use only **one <h1>** per page (the main title).
- Use headings for **structure**, not just to make text big. Search engines use them to understand the page.
- Do not skip levels (e.g. h1 directly to h4).

6. Paragraphs and Text Formatting

The **<p>** element defines a paragraph. Browsers add space before and after it. Extra spaces and new lines in the code are ignored by the browser.

```
<p>This is a paragraph.</p>
<p>First line<br>Second line (line break)</p>
<hr>                        <!-- horizontal line -->
<pre>
  Preformatted text keeps
      spaces and lines.
</pre>
```

Text formatting elements

Tag	Effect
**<b> / **	Bold / important text.
**<i> / **	Italic / emphasized text.
<u>	Underlined text.
<mark>	Highlighted text.
<small>	Smaller text.
<del> / <ins>	Deleted (strikethrough) / inserted (underlined) text.
<sub> / <sup>	Subscript (H2O) / superscript (x2).

```
<p>H<sub>2</sub>O and x<sup>2</sup>. This is <mark>highlighted</mark>.</p>
```

7. Tables

Tables display data in **rows and columns**.

Tag	Meaning
<table>	Defines the table.
<tr>	Table row.
<th>	Table header cell (bold, centered).
<td>	Table data cell.
<caption>	Title of the table.
<thead> <tbody> <tfoot>	Group header, body, footer rows.
colspan / rowspan	Merge cells across columns / rows.

```
<table border="1">
  <caption>Student Marks</caption>
  <thead>
    <tr><th>Name</th><th>Subject</th><th>Marks</th></tr>
  </thead>
  <tbody>
    <tr><td>Ravi</td><td>HTML</td><td>85</td></tr>
    <tr><td>Anita</td><td colspan="2">Absent</td></tr>
  </tbody>
</table>
```

Note: Use CSS (border, padding, border-collapse) instead of the old border attribute to style tables in real projects.

8. Dropdown (Select List)

A **dropdown list** is created with **<select>** containing **<option>** items. It is used inside forms.

```
<label for="city">Choose a city:</label>
<select id="city" name="city">
  <option value="">-- Select --</option>
  <option value="mathura" selected>Mathura</option>
```

```
  <option value="agra">Agra</option>
  <option value="delhi" disabled>Delhi</option>
</select>
```

Attribute	Use
name	Name sent to server with the value.
selected	Option selected by default.
multiple	Allows selecting more than one option.
size	Number of visible options.
disabled	Option/list cannot be selected.

Option groups and datalist

```
<select name="car">
  <optgroup label="Indian">
    <option>Tata</option><option>Mahindra</option>
  </optgroup>
  <optgroup label="Japanese">
    <option>Toyota</option><option>Honda</option>
  </optgroup>
</select>

<input list="browsers" name="browser">
<datalist id="browsers">
  <option value="Chrome"><option value="Firefox"><option value="Edge">
</datalist>
```

9. Quotations

Tag	Use
<q>	Short inline quotation; browser adds quotation marks.
<blockquote>	Long quotation; shown as an indented block. cite attribute holds the source URL.
<cite>	Title of a work (book, movie, article).
<abbr>	Abbreviation; title attribute shows full form.
<address>	Contact information of author/owner.
<bdo>	Bi-directional override: dir="rtl" reverses text direction.

```
<p>He said: <q>HTML is easy to learn.</q></p>

<blockquote cite="https://www.w3.org">
  The Web is for everyone.
</blockquote>

<p>The <abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>

<address>Written by Admin<br>Mathura, India</address>

<bdo dir="rtl">This text is reversed</bdo>
```

10. Lists

List type	Tags
Unordered	with items; shows bullets.
Ordered	with items; shows numbers/letters.
Description	<dl> with <dt> (term) and <dd> (description).

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>

<ol type="A" start="3">     <!-- type: 1, A, a, I, i -->
  <li>First</li>
  <li>Second</li>
</ol>

<dl>
  <dt>HTML</dt>
  <dd>Structure of a web page</dd>
  <dt>CSS</dt>
  <dd>Style of a web page</dd>
</dl>
```

Nested list

```
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
  <li>Backend</li>
</ul>
```

- **ul** bullet styles (CSS list-style-type): disc, circle, square, none.
- **ol** attributes: type, start, reversed.

11. Block Elements: div and span

** ** is a generic **block-level** container used to group sections of a page for layout and styling. **** is a generic **inline** container used to style a small part of text.

```
<div class="box" style="background:#eef; padding:10px;">
  <h3>Title</h3>
  <p>My favourite colour is <span style="color:red;">red</span>.</p>
</div>
```

Feature	div vs span
div	Block-level; starts new line; full width; can contain other blocks.
span	Inline; no new line; only as wide as its content.

> **Note:** Prefer semantic tags (header, section, footer) when possible; use div/span when no semantic tag fits.

12. Layout

A web page layout normally has: **header, navigation, content area(s), sidebar, footer**. HTML5 semantic tags create the structure and **CSS** positions them.

Method	Description
`Semantic tags`	header, nav, main, section, aside, footer give structure.
`CSS Float`	Older method; floats elements left/right.
`CSS Flexbox`	One-dimensional (row or column) modern layout.
`CSS Grid`	Two-dimensional (rows + columns) modern layout.

```
<style>
  .wrap { display:flex; gap:10px; }
  nav   { width:20%; background:#ddd; }
  main  { flex:1;    background:#eee; }
  aside { width:20%; background:#ccc; }
</style>

<header>Header</header>
<div class="wrap">
  <nav>Menu</nav>
  <main>Main content</main>
  <aside>Sidebar</aside>
</div>
<footer>Footer</footer>
```

13. Responsive Web Design

A **responsive** page looks good on all devices (mobile, tablet, desktop). It adapts to the screen size.

- Add the **viewport meta tag** in <head>.
- Use **relative units** (%, em, rem, vw) instead of fixed pixels.
- Make images flexible: **max-width:100%; height:auto;**
- Use **CSS media queries** to change the layout at different widths.
- Use <picture> / srcset to load different images for different screens.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<style>
  img { max-width:100%; height:auto; }
  .col { width:33%; float:left; }
  @media (max-width: 600px) {
    .col { width:100%; }   /* stack columns on mobile */
  }
</style>

<picture>
  <source media="(max-width:600px)" srcset="small.jpg">
  <img src="large.jpg" alt="Responsive image">
</picture>
```

14. iframes

An **<iframe>** (inline frame) embeds another web page inside the current page.

```
<iframe src="https://www.example.com"
        width="600" height="400"
        title="Example site"
        loading="lazy"></iframe>
```

Attribute	Use
src	URL of the page to embed.
width / height	Size of the frame.
title	Description for accessibility (required).
name	Name used as a link target.
sandbox	Applies security restrictions to embedded content.
loading="lazy"	Loads the frame only when near the screen.
allowfullscreen	Allows full screen (e.g. YouTube video).

Link target inside an iframe

```
<iframe src="page1.html" name="myframe"></iframe>
<a href="page2.html" target="myframe">Open page 2 in frame</a>
```

Note: Some websites block embedding using X-Frame-Options / CSP headers.

15. The Head Element

The **<head>** element contains **metadata** (information about the page). Its content is not displayed on the page.

Tag	Purpose
<title>	Page title shown on browser tab and search results.
<meta>	Metadata: charset, description, keywords, author, viewport.
<link>	Links external resources such as CSS files and icons.
<style>	Internal CSS.
<script>	JavaScript code or file.
<base>	Base URL / default target for all links.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Free HTML notes for students">
  <meta name="author" content="Student">
  <title>HTML Notes</title>
  <link rel="stylesheet" href="style.css">
  <link rel="icon" href="favicon.ico">
  <style> body { margin:0; } </style>
  <script src="app.js" defer></script>
</head>
```

16. Entities and URI / URL Encoding

HTML Entities

Some characters are reserved in HTML (like < and >). To display them, or to show symbols not on the keyboard, use **entities**. Syntax: **&entity_name;** or **&#entity_number;**

Entity	Result / Meaning
** **	Non-breaking space
< >	< and >
&	& (ampersand)
"	" (double quote)
'	' (apostrophe)
©	© Copyright
®	® Registered
™	™ Trademark
€	€ Euro
₹	₹ Indian Rupee

```
<p>5 &lt; 10 &amp;amp;&amp;amp; 10 &gt; 2</p>
<p>&copy; 2026 My Company&trade;</p>
```

URL / URI and URL Encoding

URI (Uniform Resource Identifier) identifies a resource. **URL** (Locator) is a URI that also tells where it is. Structure of a URL:

```
scheme://host:port/path?query#fragment

https://www.example.com:443/products/list.html?id=5&sort=asc#top
```

URLs can contain only ASCII characters. Other characters are **URL-encoded** as **% + hexadecimal value**.

Character	Encoded
space	%20 (or + in query strings)
!	%21
#	%23
$	%24
&	%26
/	%2F
?	%3F
=	%3D
@	%40

17. Frames (Old HTML)

Frames divided the browser window into several independent sections, each showing a different HTML file, using <frameset> and <frame>.

```
<!-- OBSOLETE in HTML5 - for understanding only -->
<frameset cols="25%,75%">
  <frame src="menu.html">
  <frame src="content.html">
  <noframes><body>Your browser does not support frames.</body></noframes>
</frameset>
```

Tag / attribute	Meaning
<frameset>	Container that defines frames (replaces body).
cols / rows	Divide the window vertically / horizontally.
<frame>	One frame with src file.
<noframes>	Text for browsers without frame support.

> **Note:** <frameset> and <frame> are **not supported in HTML5**. Use **<iframe>** or CSS layouts instead.

18. Charset (Character Encoding)

A **character set** tells the browser how to convert stored numbers into readable characters. Without it, text may appear as strange symbols.

Charset	Description
ASCII	128 characters (English letters, digits, symbols).
ISO-8859-1	Latin-1; 256 characters, Western European.
UTF-8	Unicode; supports almost all languages and emojis. **Default and recommended.**

```
<head>
  <meta charset="UTF-8">   <!-- must be within first 1024 bytes -->
</head>
```

> **Note:** Save your file in UTF-8 format as well, otherwise Hindi/emoji text may not display correctly.

19. Forms

A **form** collects input from the user and sends it to a server for processing. Created with the **<form>** element.

```
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>
  <input type="submit" value="Send">
</form>
```

Attribute	Use
action	URL of the server page that processes the data.
method	GET or POST: how data is sent.

Attribute	Use
`enctype`	Encoding; use multipart/form-data for file upload.
`target`	Where to show the response (_self, _blank).
`autocomplete`	on / off: browser auto-fill.
`novalidate`	Skip browser validation.

GET vs POST

	GET	POST
`Data location`	Visible in URL	Sent inside request body
`Security`	Less secure	Safer for passwords/private data
`Data size`	Limited	No practical limit
`Use for`	Search, filters	Login, registration, uploads

20. Form Elements

Element	Purpose
`<input>`	Most common field; behaviour depends on type attribute.
`<textarea>`	Multi-line text box (rows, cols).
`<select>`	Dropdown list.
`<button>`	Clickable button.
`<label>`	Caption linked to an input using for=id.
`<fieldset> <legend>`	Groups related fields with a title.

Input types

type	Description
`text`	Single-line text.
`password`	Hidden characters.
`email / url / tel`	Validated email, web address, phone.
`number / range`	Number field / slider.
`date / time`	Date and time pickers.
`checkbox`	Select zero or more options.
`radio`	Select only one option from a group (same name).
`file`	Upload a file.
`color`	Colour picker.
`hidden`	Invisible field to send data.
`submit / reset / button`	Send form / clear form / generic button.

Useful input attributes

- **placeholder** (hint text), **required**, **readonly**, **disabled**, **value**, **maxlength**, **min / max**, **pattern** (regex), **autofocus**.

Complete example

```
<form action="/register" method="post">
  <fieldset>
    <legend>Registration</legend>

    <label for="u">Username:</label>
    <input type="text" id="u" name="user" placeholder="Enter name" required>

    <label for="e">Email:</label>
    <input type="email" id="e" name="email" required>

    <label for="p">Password:</label>
    <input type="password" id="p" name="pass" minlength="8" required>

    Gender:
    <input type="radio" name="g" value="m"> Male
    <input type="radio" name="g" value="f"> Female

    Hobbies:
    <input type="checkbox" name="h" value="read"> Reading
    <input type="checkbox" name="h" value="music"> Music

    <label for="c">Course:</label>
    <select id="c" name="course">
      <option>HTML</option><option>CSS</option>
    </select>

    <textarea name="msg" rows="3" cols="30">Write here...</textarea>

    <button type="submit">Register</button>
    <button type="reset">Reset</button>
  </fieldset>
</form>
```

21. HTML Security

Web security protects users and websites from attacks. Following are the key practices for HTML developers:

- **Use HTTPS** so data between browser and server is encrypted.
- **Validate input** in the browser (required, pattern, type) **and always on the server** too. Client-side checks can be bypassed.
- **Prevent XSS (Cross-Site Scripting)**: never insert user input directly into a page; escape characters like < > & " (use entities).
- **Prevent CSRF**: include a secret token in forms so only your own site can submit them.
- **Passwords**: use type="password", send with POST, and never put sensitive data in the URL (GET).
- **External links**: add rel="noopener noreferrer" when using target="_blank".
- **iframes**: use the sandbox attribute and only embed trusted sites; protect your own site with X-Frame-Options to stop clickjacking.
- **CDN files**: use Subresource Integrity (integrity attribute) so a modified file is not loaded.

- **Content Security Policy (CSP)** limits which scripts and resources can run.
- Use autocomplete="off" on sensitive fields; avoid inline scripts.

```
<a href="https://other.com" target="_blank" rel="noopener noreferrer">Visit</a>

<iframe src="widget.html" sandbox="allow-scripts" title="Widget"></iframe>

<script src="https://cdn.example.com/lib.js"
        integrity="sha384-...hash..."
        crossorigin="anonymous"></script>

<meta http-equiv="Content-Security-Policy" content="default-src 'self'">
```